



Sistemas Distribuidos I (75.74)

Algoritmos de Consenso

Elección de Líder. Generales Bizantinos. Paxos.

Docentes

- Pablo D. Roca
- Gabriel Robles
- Franco Barreneche

- Tomás Nocetti
- Nicolás Zulaica



- **Elección de Líder**
- Consenso
- Generales Bizantinos
- Paxos



- **Objetivo**
 - Elegir a un proceso en un grupo para que desempeñe un rol particular
 - Permitir reelecciones en caso de que proceso líder decida darse de baja
 - Permitir reelecciones en caso de que proceso líder se encuentre caído
- **Características**
 - Cualquier proceso puede comenzar una nueva elección de líder
 - En ningún momento puede haber más de un líder
 - El resultado de la elección de un nuevo líder debe ser **única y repetible**



Elección de Líder | Propiedades

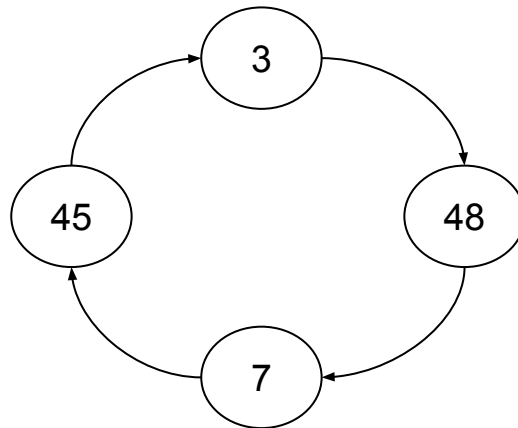
- Cada proceso debe tener un identificador único
- Todos los procesos poseen un array que indica el estado del algoritmo de elección de Líder
- Estados posibles: *Identificador (P)*, *indefinido (@)*
 - *Indefinido (@)* es el estado inicial que posee un proceso P_i al comenzar a participar del algoritmo de elección de líder
- **Safety**
 - Un proceso participante P_i posee el estado $elected_i = P$ o $elected_i = @$
- **Liveness**
 - Todos los procesos participan de la elección de líder y bien terminan con un estado $elected_i \neq @$ o comienzan una nueva elección de líder



Elección de Líder | Algoritmo del anillo

- **Condiciones Iniciales**

- Cada proceso se comunica solamente con su vecino
- Mensaje son enviados siempre en la misma dirección (por ejemplo, sentido horario)
- Al comienzo del algoritmo, todos los procesos son marcados como *no participantes*





Elección de Líder | Algoritmo del anillo

- **Inicio de la elección**

- Algún proceso P_i se marca como *participando* y envía un mensaje en sentido horario indicando que él es el líder.

- **Proceso P_j recibe mensaje de elección de líder y...**

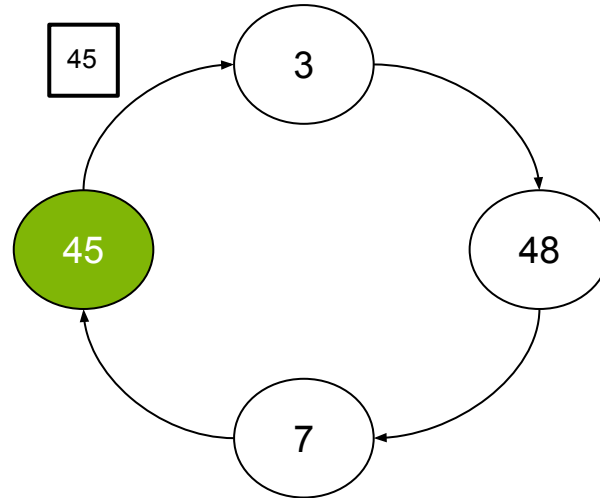
- Si se encuentra en estado *no participando*:
 - Cambia su estado a *participando*.
 - Compara el identificador del líder con el suyo, lo reemplaza en caso de ser mayor y reenvía el mensaje al nodo siguiente.
- Si se encuentra en estado *participando*:
 - Si el ID recibido es menor al suyo, no reenvía el mensaje.
 - Si el ID recibido es mayor al suyo, reenvía el mensaje.
 - Si el ID recibido es igual al suyo, entonces **es el líder!!**



Elección de Líder | Algoritmo del anillo

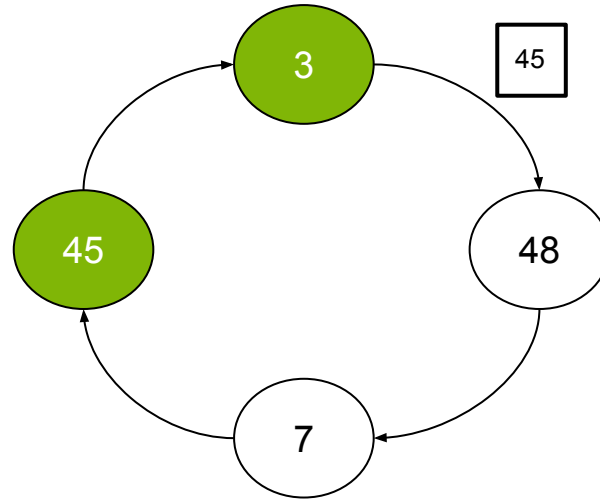
- **Proceso P_j reconoce que es el líder y...**
 - Se setea como *no participando*.
 - Envía un mensaje de **líder elegido**.
- **Proceso P_j recibe un mensaje de líder elegido y...**
 - Cambia su estado a *no participando*.
 - Setea la variable elegido con el identificador del mensaje recibido.
 - Si el identificador recibido es distinto al suyo, retransmite el mensaje.

Elección de Líder | Algoritmo del anillo



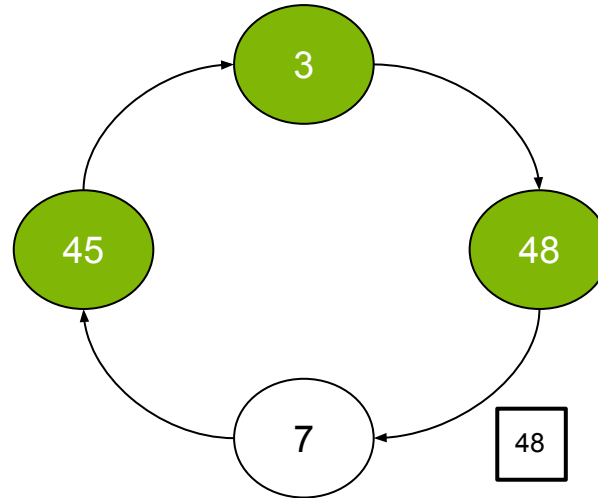
P45 se setea como participando y le envía un mensaje con su ID a P3

Elección de Líder | Algoritmo del anillo



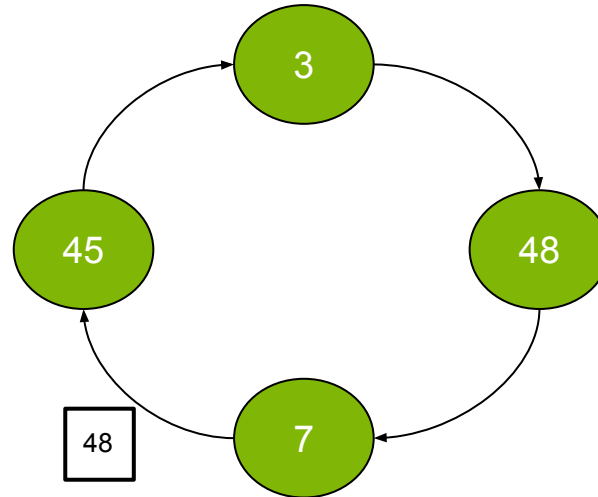
P3 se setea como participando y al ver que el mensaje recibido es de un ID mayor al suyo, lo reenvía a P48

Elección de Líder | Algoritmo del anillo



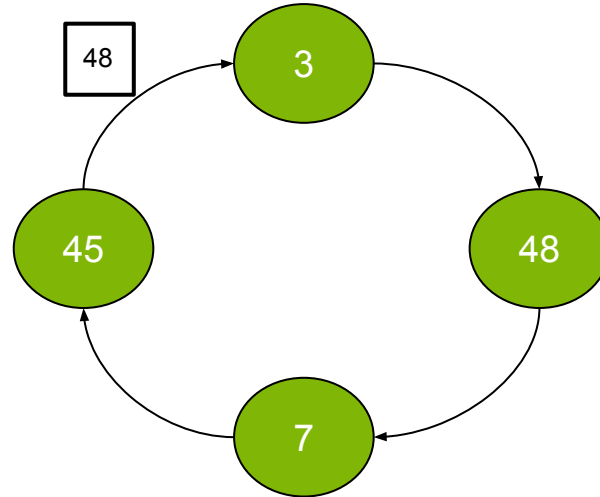
P48 se setea como participando y al ver que el mensaje recibido es de un ID menor al suyo, le envía su propio ID a P7

Elección de Líder | Algoritmo del anillo



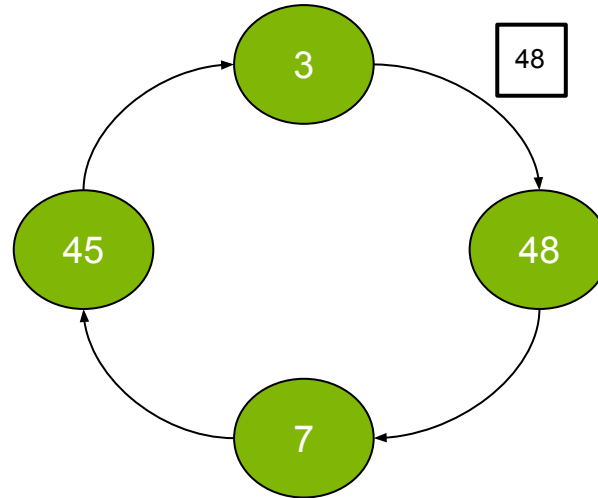
P7 se setea como participando y al ver que el mensaje recibido es de un ID mayor al suyo, lo reenvía a P45

Elección de Líder | Algoritmo del anillo

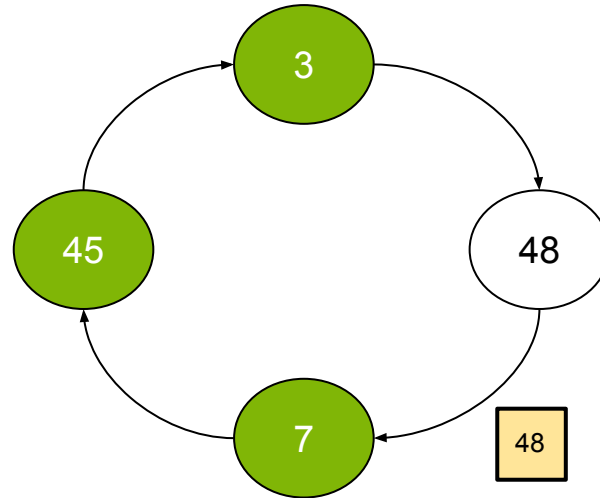


P45 (ya participando)
reenvía el mensaje a P3 ya
que el ID del mismo es
mayor al suyo

Elección de Líder | Algoritmo del anillo

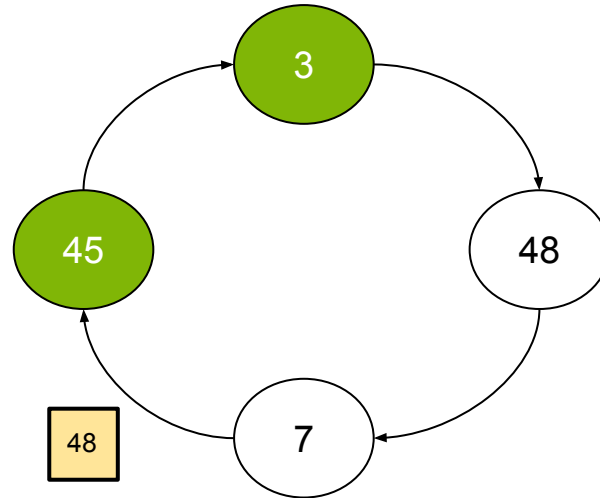


P3 (ya participando)
reenvía el mensaje a P48
ya que el ID del mismo es
mayor al suyo



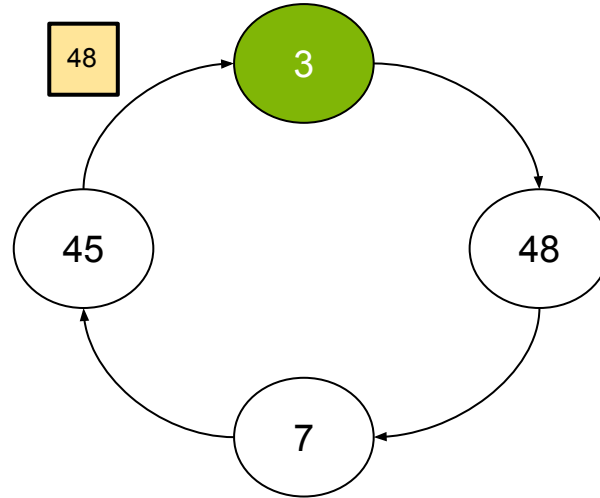
P48 (ya participando) al recibir su propio ID detecta que es el nuevo líder, por lo que setea su estado a "no participando" y le envía el mensaje de **líder elegido** a P7

Elección de Líder | Algoritmo del anillo

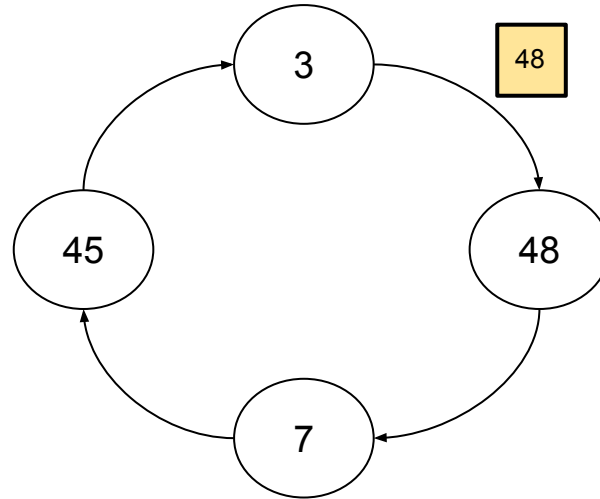


P7 (ya participando) recibe un mensaje de **líder elegido** por lo que setea su estado como "no participando" y lo reenvía a P45

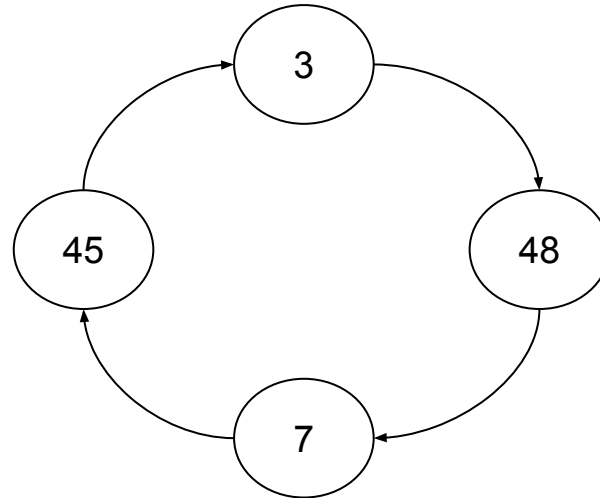
Elección de Líder | Algoritmo del anillo



P45 (ya participando)
recibe un mensaje de
líder elegido por lo que
setea su estado como "no
participando" y lo reenvía
a P3



P3 (ya participando) recibe un mensaje de **líder elegido** por lo que se setea como "no participando" y lo reenvía a P48



P48 (no participando)
recibe un mensaje de
líder elegido con su
propio ID por lo que lo
procede a dropearlo y
finalizar la elección



Elección de Líder | Bully Algorithm

- **Hipótesis**
 - Canales de comunicación *reliables*
 - Cualquier proceso puede morir de forma inesperada
 - Uso de timeouts para detectar que un proceso no está respondiendo
 - Todos los procesos pueden comunicarse entre sí
 - Cada proceso conoce el ID asociado a todos los procesos
 - Cada proceso conoce qué procesos **poseen un ID superior al suyo**



Elección de Líder | Bully Algorithm

- **Tipos de Mensajes**

- Election Message: Mensaje enviado para iniciar una elección de líder
- Answer Message: ACK de Election Messages
- Coordinator Message: Contiene información sobre quién fue el proceso líder elegido

- **Sincronismo**

- T_{max}: Tiempo máximo de transmisión
- T_{process}: Tiempo máximo de cualquier proceso para resolver un mensaje
- **$T = 2 * T_{max} + T_{process}$** : timeout para detectar procesos caídos.



Elección de Líder | Bully Algorithm

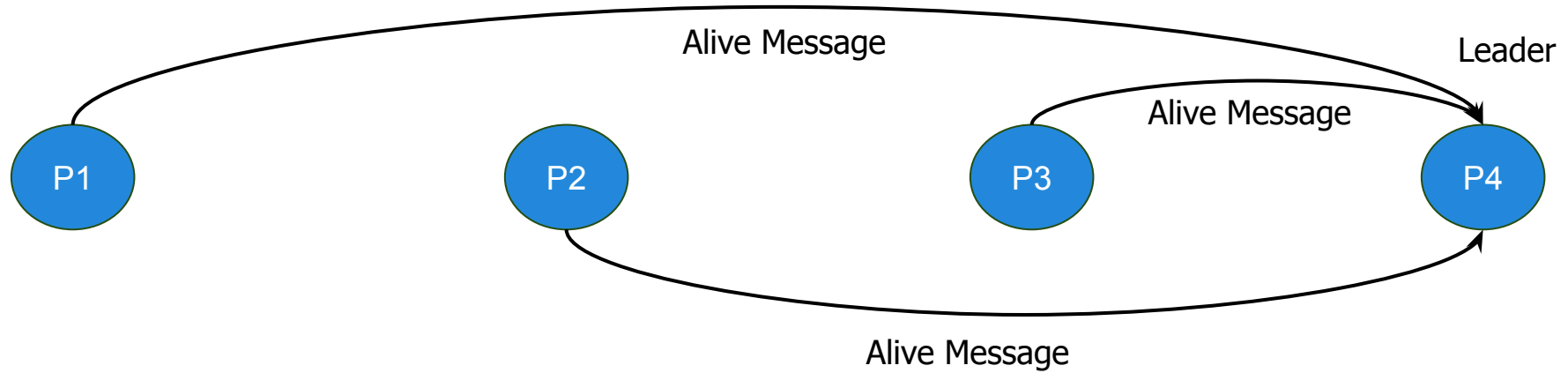
- **Algoritmo (Parte I)**

- Proceso con mayor ID puede identificarse como leader y mandar un *Coordinator Message* a todos los procesos del sistema
- Si un proceso detecta que el líder está caído, envía *Election Messages* a procesos que tengan **un ID mayor al suyo**
- Si un proceso recibe un *Election Message*, responde con un *Answer Message* y **comienza una nueva elección**
- Si un proceso recibe un *Coordinator Message*, elige al proceso que envió el mensaje como líder
- Si un proceso que comenzó una elección no recibe *Answer Messages* después de transcurrido tiempo T, **el proceso se autoproclama líder**



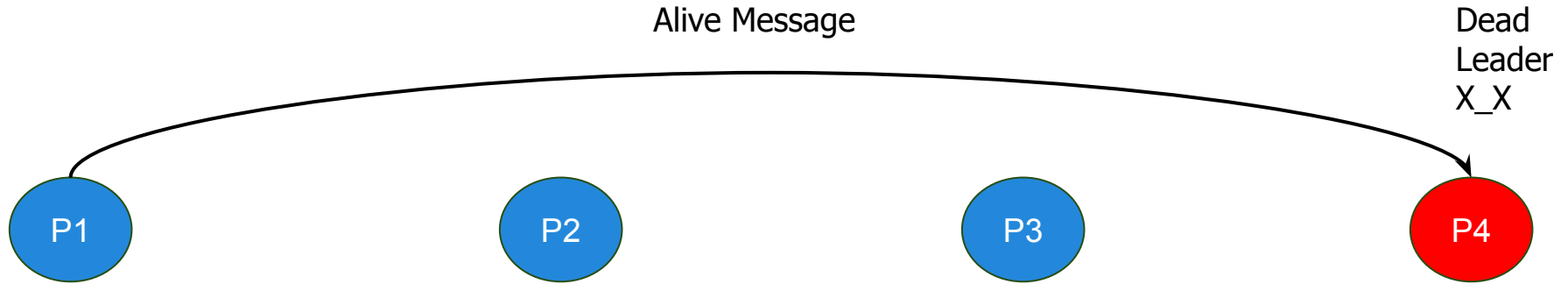
- **Algoritmo (Parte II)**
 - Si un proceso caído vuelve a la vida, **comienza una nueva elección de líder**
 - Si este proceso es el que posee mayor ID, será elegido como el nuevo líder
 - Por esta razón este algoritmo es llamado **Bully Algorithm**

Elección de Líder | Bully Algorithm



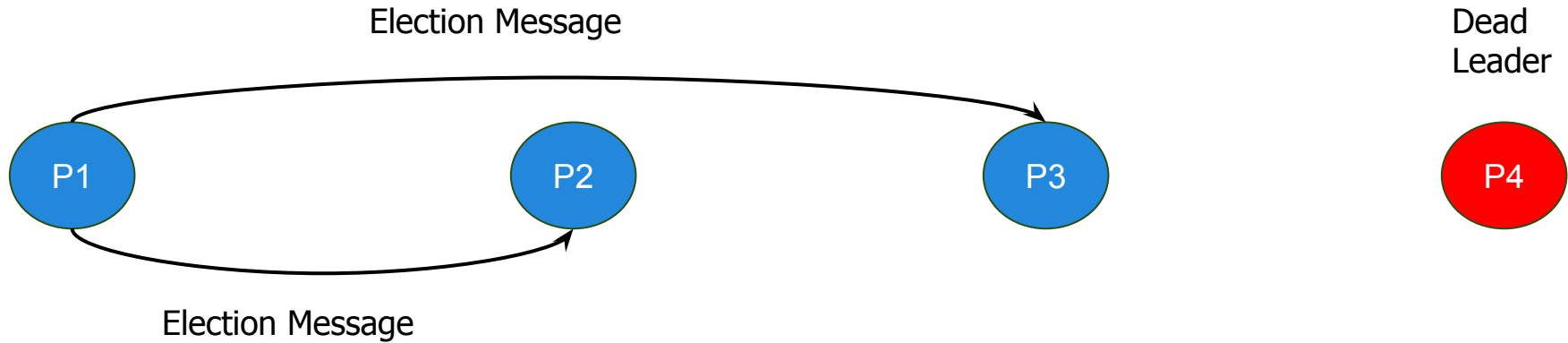
Periódicamente se verifica el estado del líder con AliveMessages.

Elección de Líder | Bully Algorithm



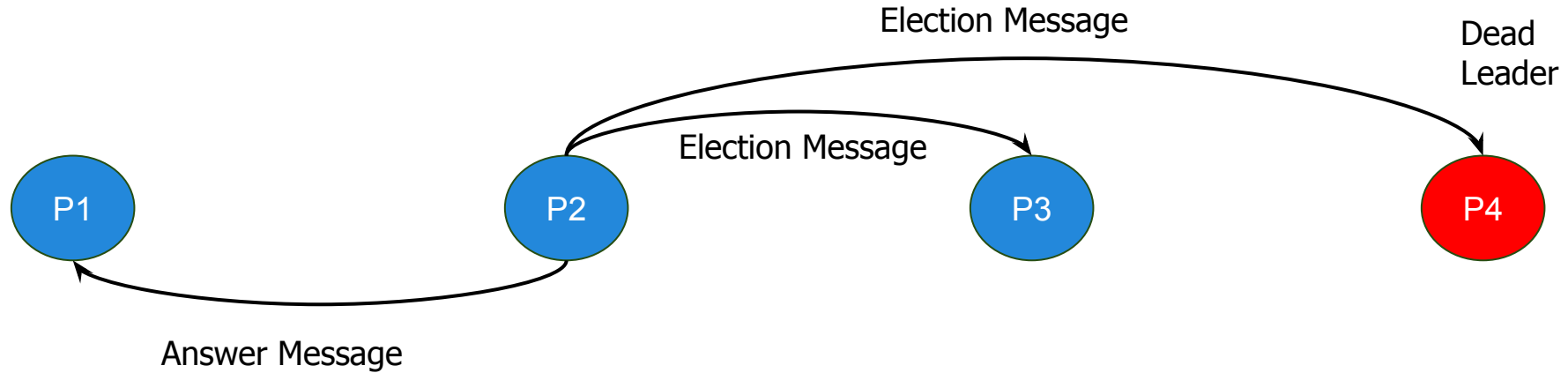
P1 detecta que P4 está caído al no responderle un AliveMsg.

Elección de Líder | Bully Algorithm



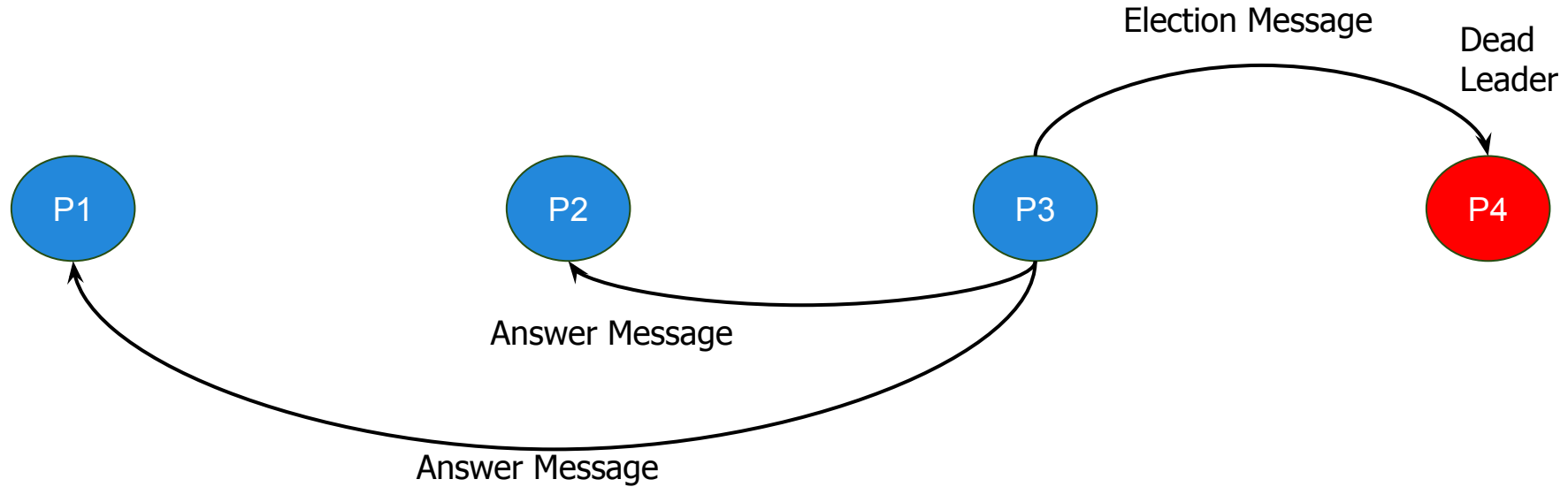
P1 envía un ElectionMsg tanto a P2 como a P3.

Elección de Líder | Bully Algorithm



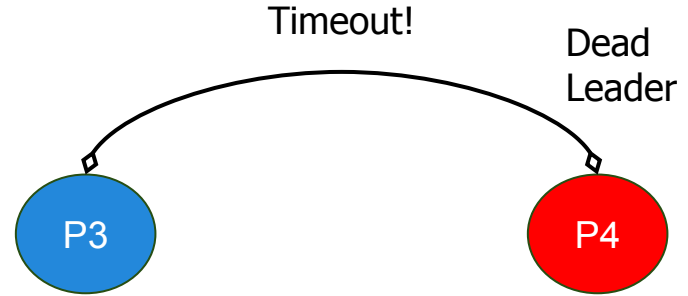
P2 responde a P1 con un AnswerMsg y envía otro ElectionMsg a P3.

Elección de Líder | Bully Algorithm



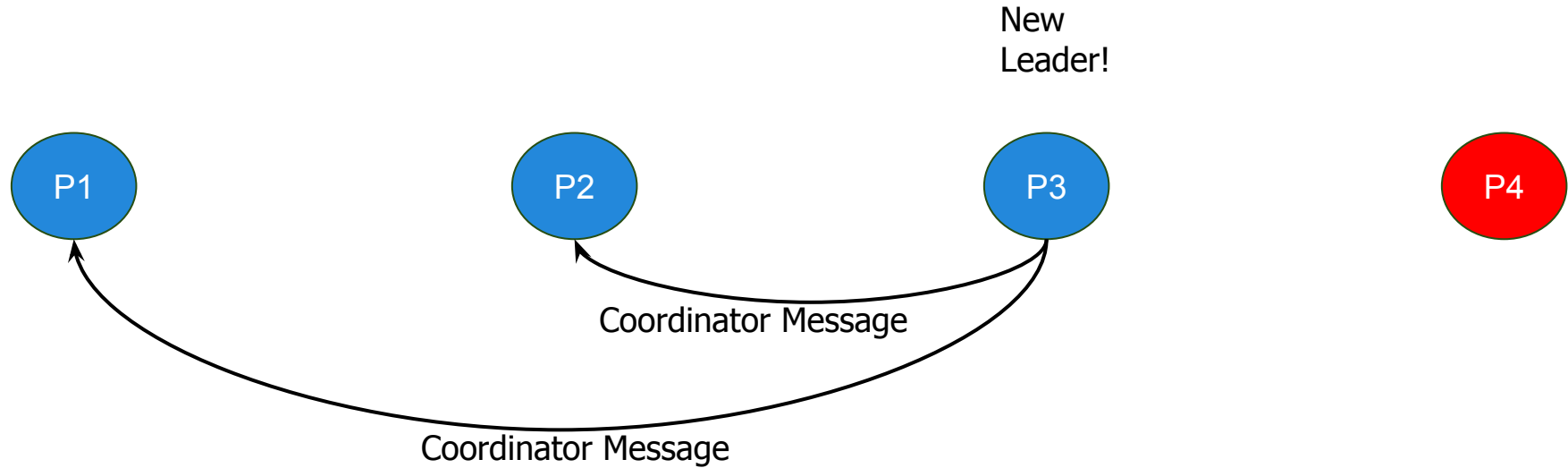
P3 responde a P1 y P2 con un AnswerMsg y envía otro ElectionMsg a P4.

Elección de Líder | Bully Algorithm



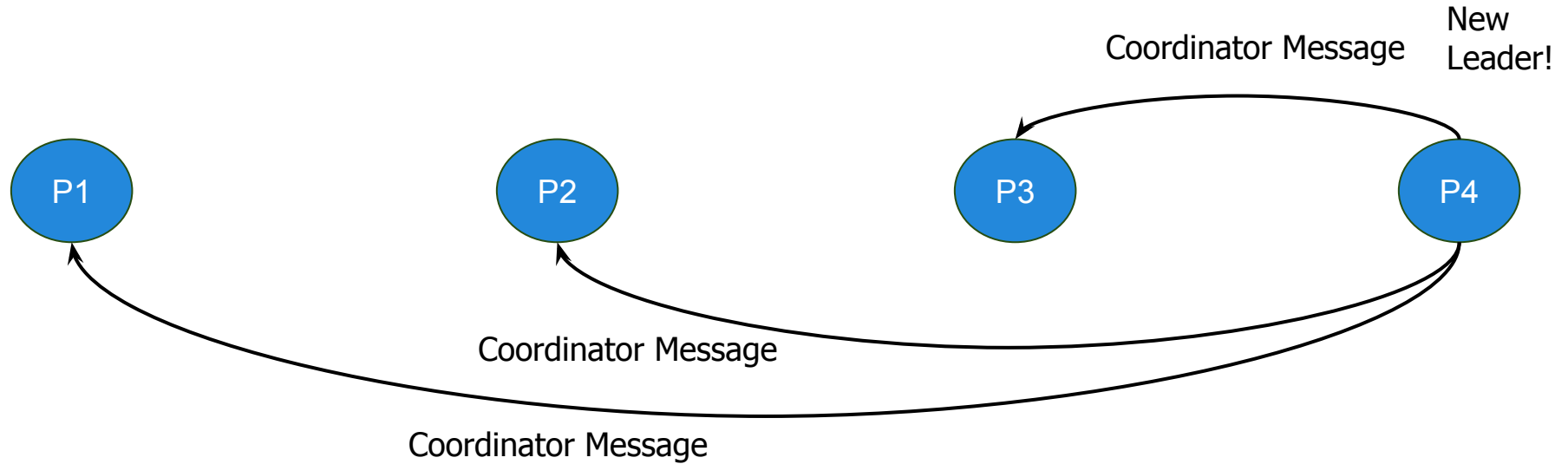
P4 al estar caído no responde y dispara un timeout en P3.

Elección de Líder | Bully Algorithm



P3 se autoproclama líder y lo informa con un CoordinatorMsg a P1 y P2.

Elección de Líder | Bully Algorithm



P4 vuelve a la vida! Sabe que su ID es más alto y se autoproclama líder.

Agenda



- ☐ Elección de Líder
- ☒ **Consenso**
- ☐ Generales Bizantinos
- ☐ Paxos



- Dado un conjunto de procesos distribuidos y un punto de decisión, todos los procesos deben acordar en el mismo valor
- **Problema complejo, requiere acotar variables:**
 - Canales de comunicación son *reliables*
 - Todos los procesos pueden comunicarse entre sí
 - Única falla a considerar es la caída de un proceso
 - Caída de un proceso no puede ocasionar la caída de otro
- Propiedades necesarias de los algoritmos de consenso:
 - *Agreement*
 - *Integrity*
 - *Termination*



Consenso | Definición y Requerimientos del Problema

- **Definición**

- Conjuntos de P_i ($i = 1 \dots N$) procesos desean llegar a un acuerdo
- Cada proceso comienza en el estado ***undecided***
- Cada proceso posee una ***decision variable*** d_i ($i = 1 \dots N$)
- Cada proceso propone un valor v_i
- Procesos se comunican entre sí a través de mensajes
- Luego de haber recibido mensajes de otros procesos, proceso P_i setea su ***decision variable*** d_i y cambia su estado a ***decided***



Consenso | Definición y Requerimientos del Problema

- **Requerimientos**

- *Agreement*

- El valor de la variable es el mismo en todos los procesos correctos
 - Si P_i y P_j son procesos correctos entonces $d_i = d_j$ cuando su estado es ***decided***

- *Integrity*

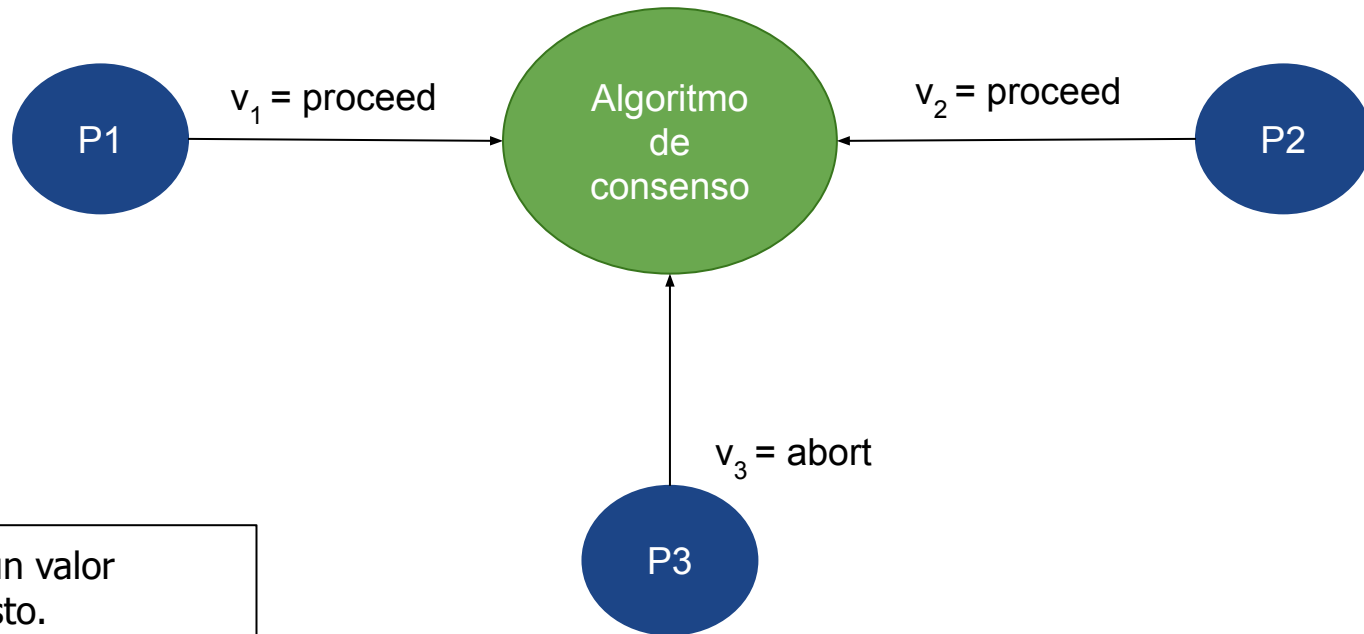
- Si los procesos correctos propusieron el mismo valor v_i , entonces el valor de su ***decision variable*** es la misma

- *Termination*

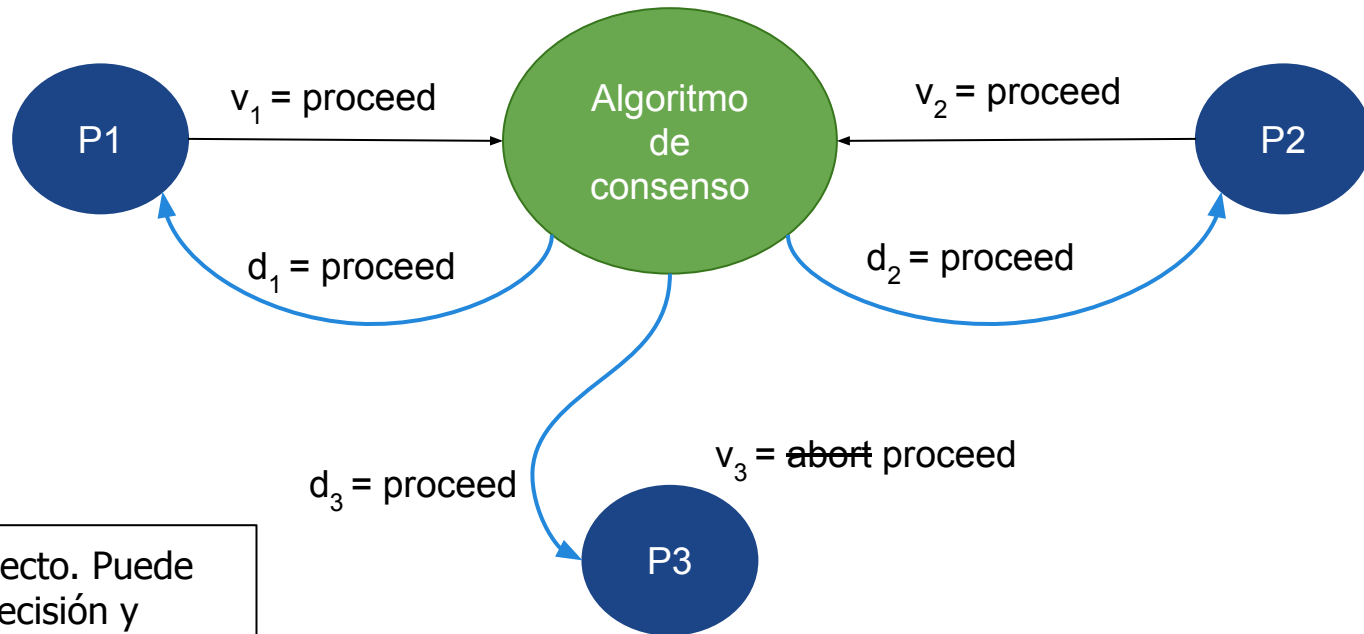
- Eventualmente todos los procesos activos setean su ***decision variable***

- **Fórmula de Quorum**

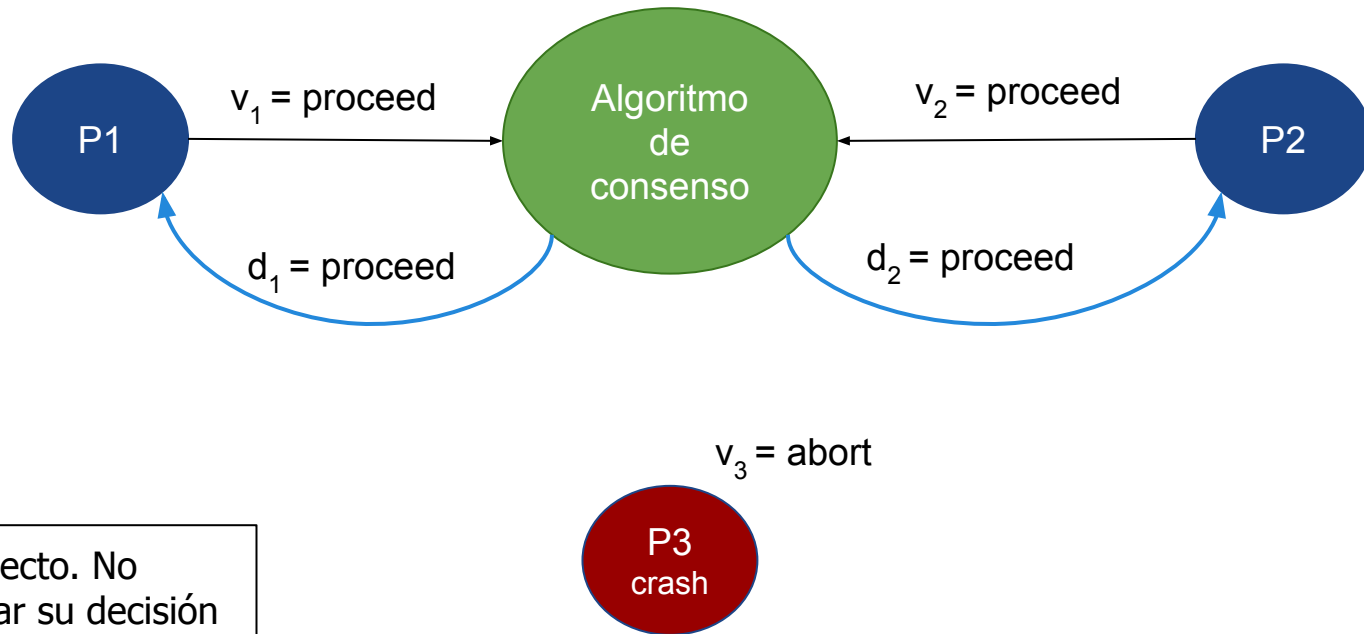
- Depende de la fórmula de agregación. Ej. si requiere mayoría: $N \geq 2f + 1$.



P3 propone un valor distinto al resto.



P3 no es correcto. Puede cambiar de decisión y seguir al consenso.



P3 no es correcto. No puede cambiar su decisión y se declara defectuoso.



Consenso | Algoritmo sincrónico

Init

Values(i , 0) = {}, Values(i , 1) = { v }

For each round r , $1 < r \leq f+1$

 broadcast Values(i , r) - Values(i , $r-1$)

 Values(i , $r+1$) = Values(i , r)

 While round r still open

 receive Values from j in Values(j , r)

 Values(i , $r+1$) = Values(i , $r+1$) $\cup$ Values(j , r)

After $f+1$ rounds

 decide d = aggregation over Values(i , $f+1$)

Agenda



- ☐ Elección de Líder
- ☐ Consenso
- ☒ **Generales Bizantinos**
- ☐ Paxos



Generales Bizantinos | Definición y *Requerimientos*

- **Definición**

- Tres o más **generales** deben decidir si atacan o se retiran
- Un **comandante** envía la orden de ataque/retirada
- **Generales** pueden ser *traicioneros*
 - Le indica a los otros generales que deben atacar si el comandante le indicó que se retiren y viceversa
- **Comandante** puede ser *traicionero*
 - Envía diferentes órdenes a diferentes generales
- **Comandante / General** *traicionero* viene a emular a un proceso que posee fallas



Generales Bizantinos | Definición y *Requerimientos*

- **Requerimientos**

- *Agreement*

- El valor de la variable ***decided*** es el mismo en todos los procesos activos
 - Si P_i y P_j son procesos activos entonces $d_i = d_j$ cuando su estado es ***decided***

- *Integrity*

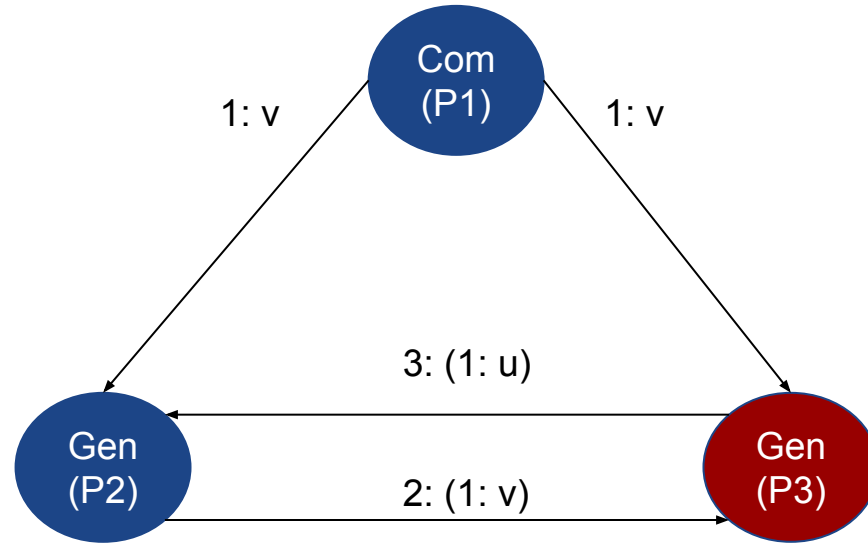
- Si el comandante *no es traicionero*, todos los procesos deben setear su ***decision variable*** al valor enviado por el comandante

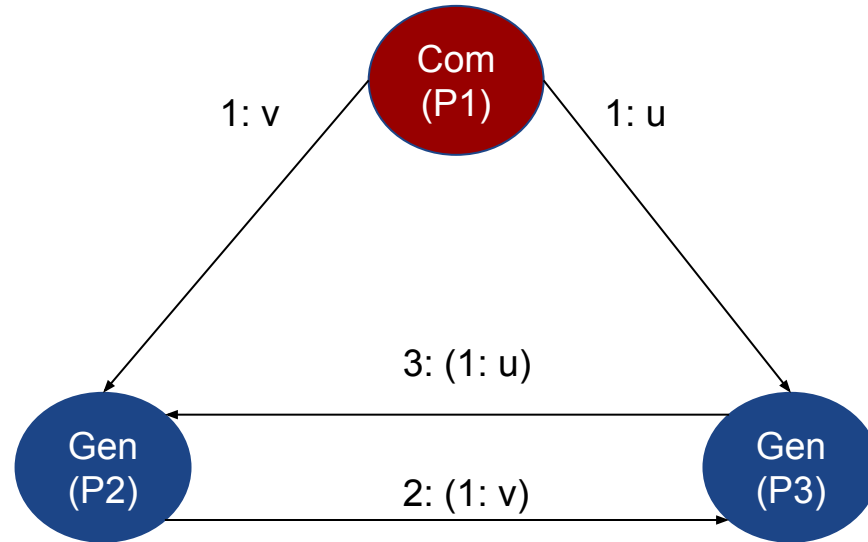
- *Termination*

- Eventualmente todos los procesos activos setean su ***decision variable***

- ***Fórmula de Quorum***

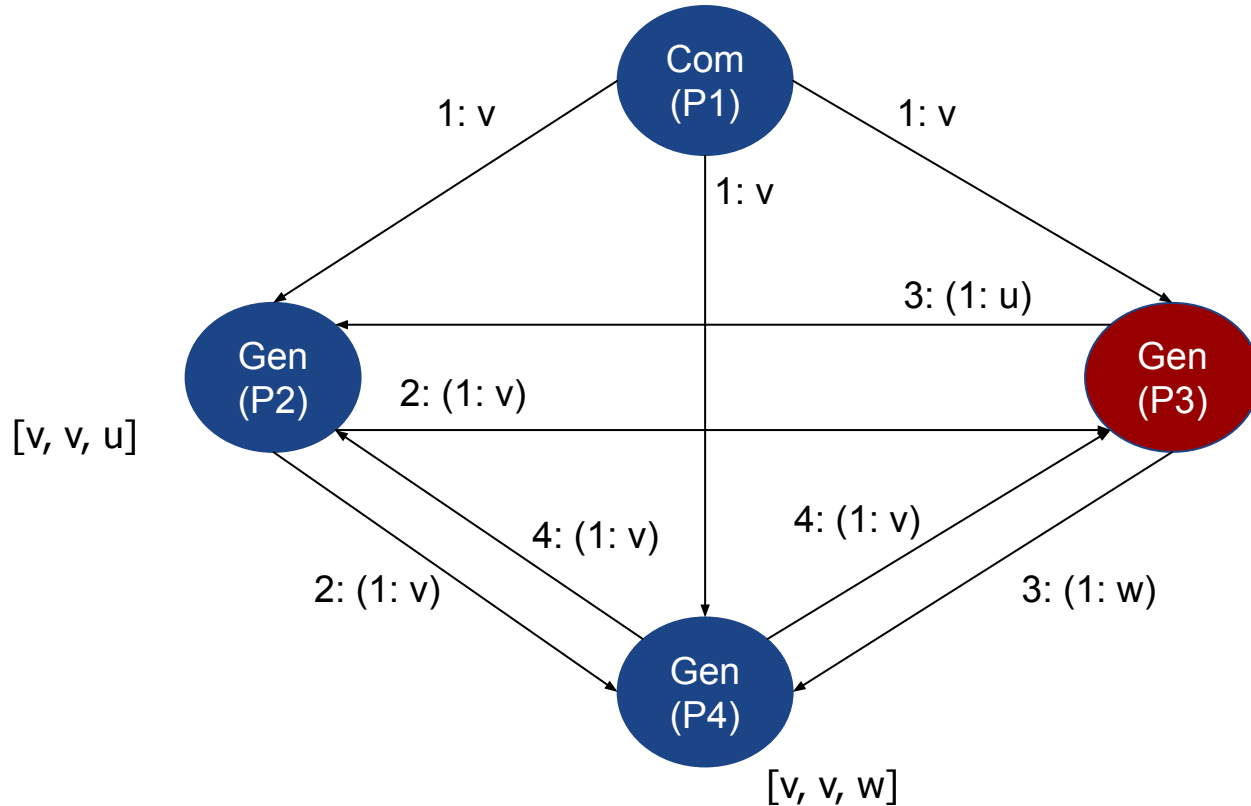
- $N \geq 3f + 1$





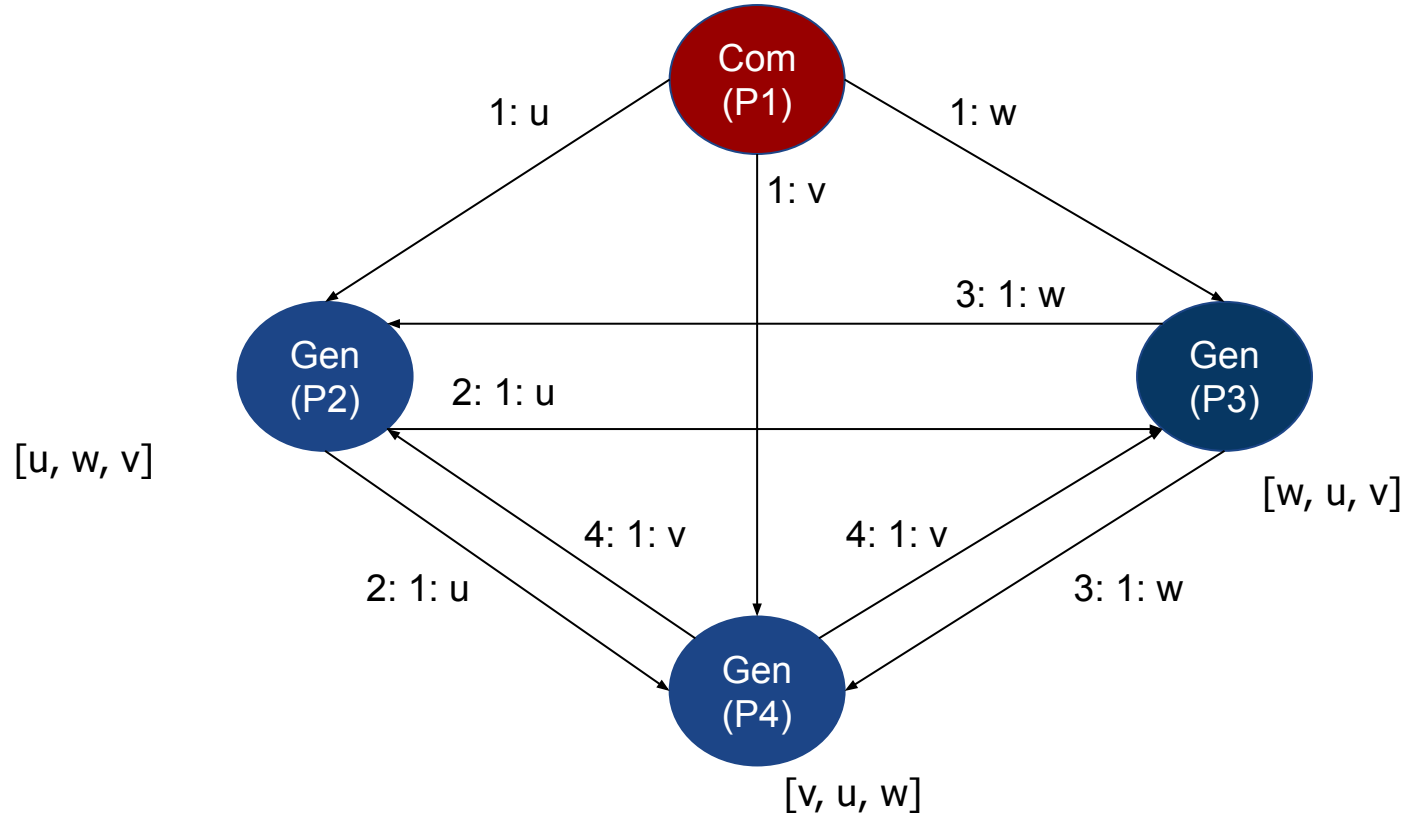


Generales Bizantinos | Solución $N \geq 3f + 1$





Generales Bizantinos | Solución $N \geq 3f + 1$



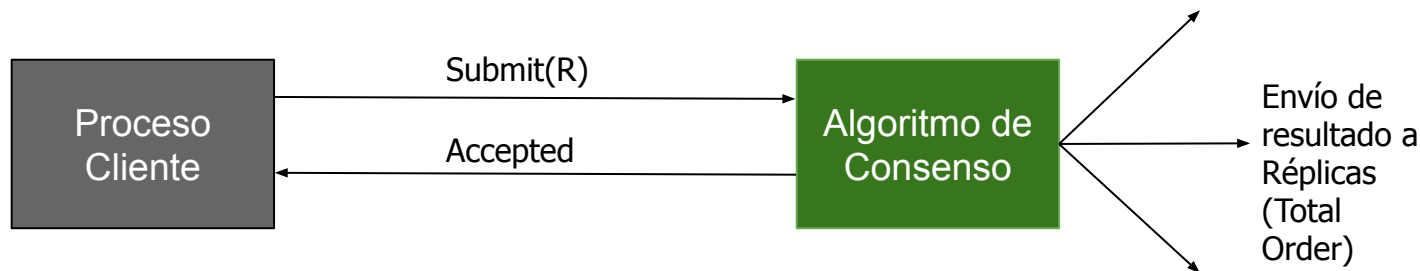
Agenda



- ☐ Elección de Líder
- ☐ Consenso
- ☐ Generales Bizantinos
- ☒ **Paxos**



- **Objetivo**
 - Consensuar un valor aunque múltiples procesos realicen diferentes propuestas
- **Características**
 - **Tolerante a Fallos**
 - Algoritmo progresa siempre que haya una mayoría de procesos vivos
 - $N \geq 2f + 1$ (Fórmula de Quorum)
 - **Posible Rechazar Propuestas**
 - Pedido de un cliente puede ser rechazado
 - Cliente puede reintentar una propuesta las veces que desee

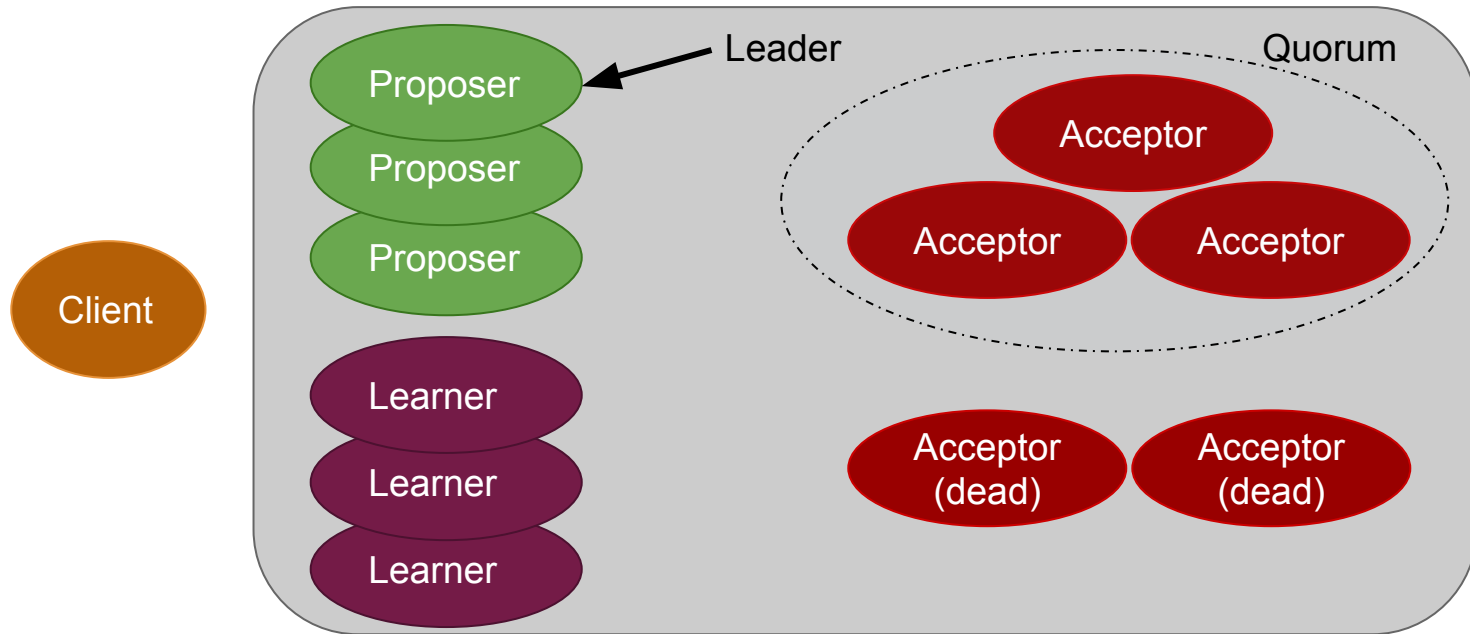


- Cliente intenta setear/modificar un valor R (key/value)
 - Si puede hacerlo, recibe un OK
 - En caso contrario, debe volver a intentar
- **Paxos asegura orden consistente en un cluster**
 - Eventos realizados por clientes son almacenados de forma incremental por ID



Paxos | Arquitectura

- **Objetivo:** Lograr que todos los **Acceptors** consensuen un valor **v** asociado a una propuesta realizada por un **Proposer**

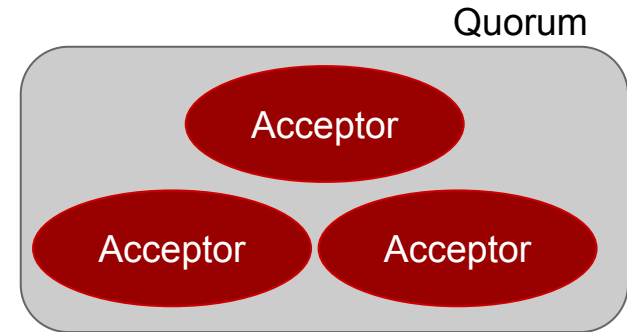
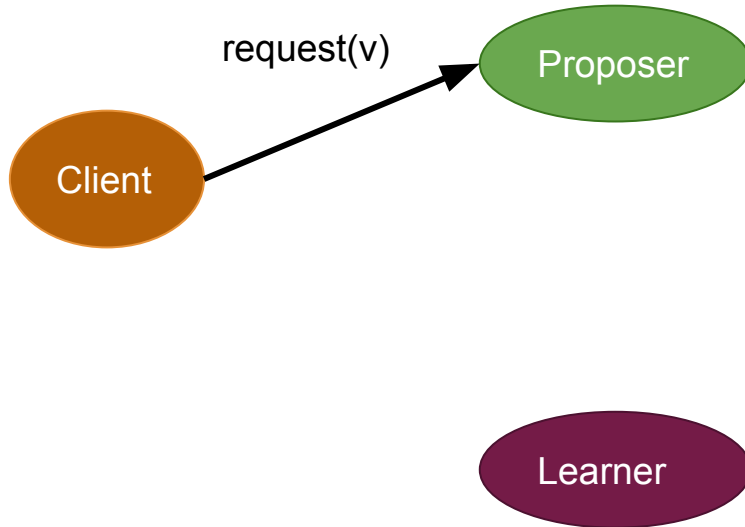




- **Proposer**
 - Reciben requests de clientes y comienzan el protocolo
 - *Leader* debe ser elegido para evitar *starvation*
- **Acceptor**
 - Reciben prepare/propose de los Proposers
 - Mantienen el estado del protocolo en almacenamiento estable
 - Quorum: Existe cuando la mayoría de Acceptors se encuentran vivos
- **Learner**
 - Cuando el protocolo llega a un acuerdo, Learner ejecuta el request y envía respuesta al cliente



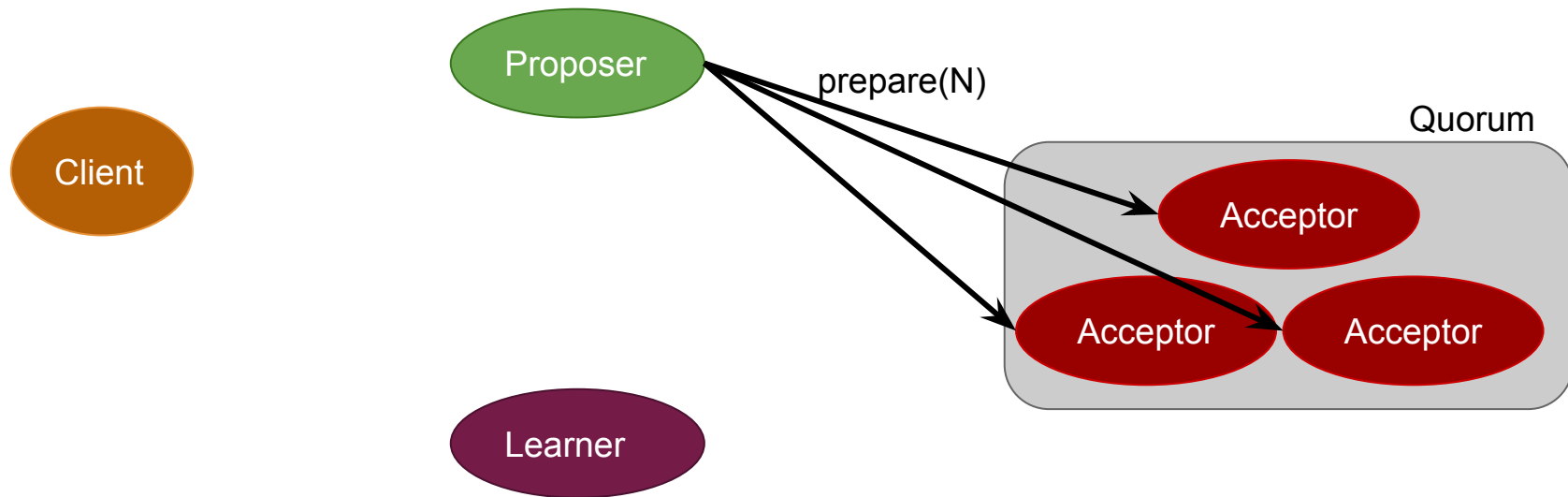
- **Client** realiza un Request





Paxos | Fase 1a - Prepare

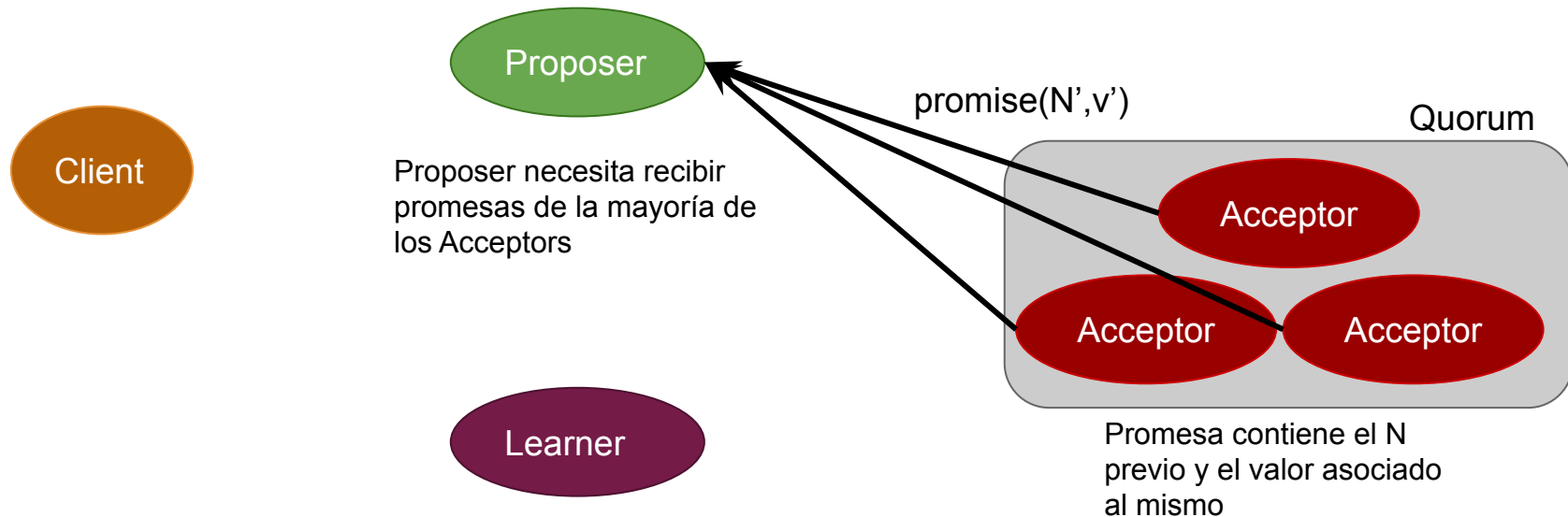
- **Proposer:** Crea una propuesta #N donde N es mayor a cualquier propuesta realizada previamente por el Proposer
- Enviar propuesta a Acceptors esperando obtener Quorum (mensaje llegue a la mayoría de Acceptors)





Paxos | Fase 1b - Promise

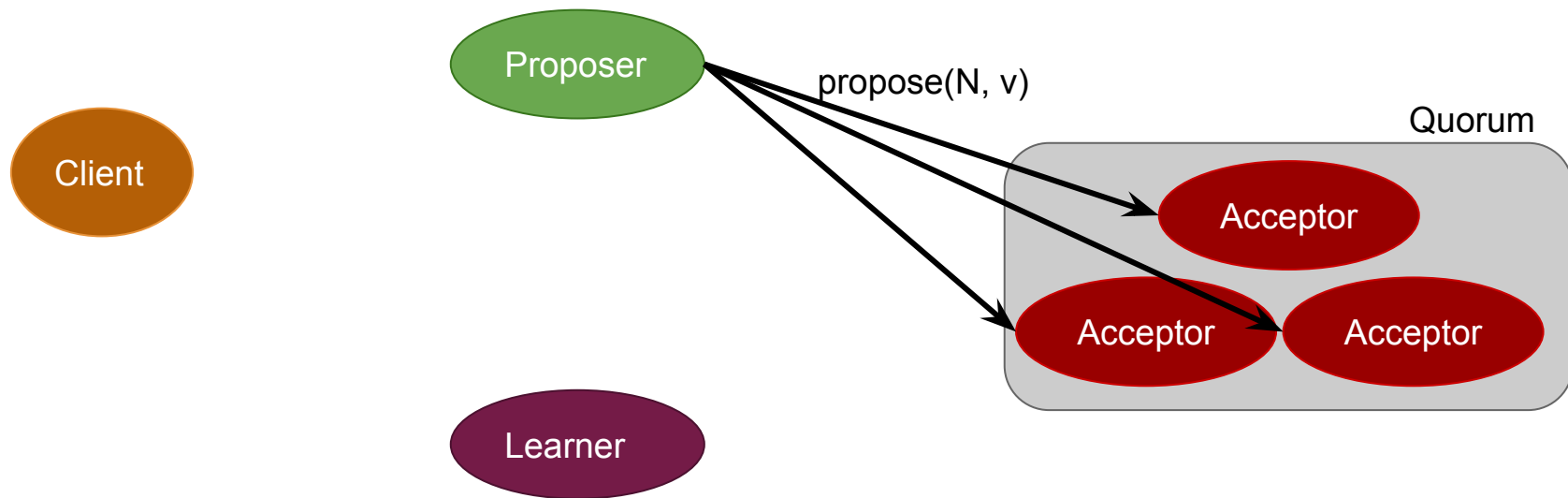
- Si ID recibido del Proposer es mayor al último recibido, Acceptors prometen rechazar cualquier Requests con $ID < N$
- Envío de Promesa a Proposer
- Acceptors no contestan si llega una propuesta que no cumpla que $N > N'$





Paxos | Fase 2a - Propose

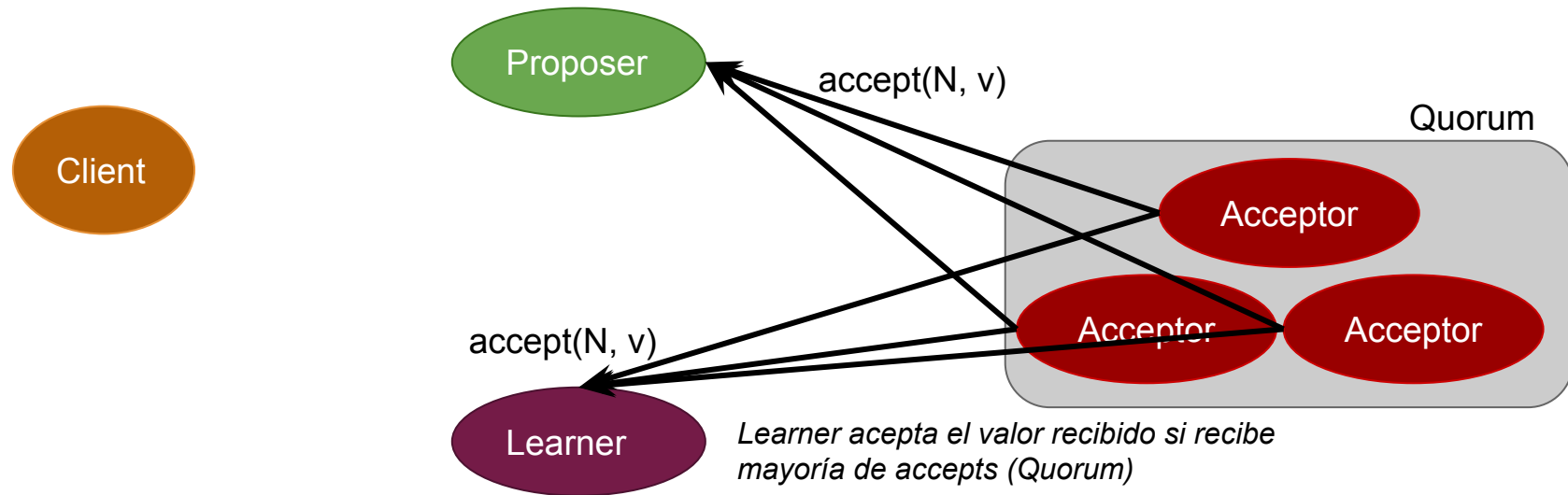
- **Proposer:** Si recibe Promesas de la mayoría:
 - Rechaza todos los requests con un ID $< N$
 - Envía Propose con el N recibido por los acceptors y un valor v: $\text{Propose}(N, v)$





Paxos | Fase 2b - Accept

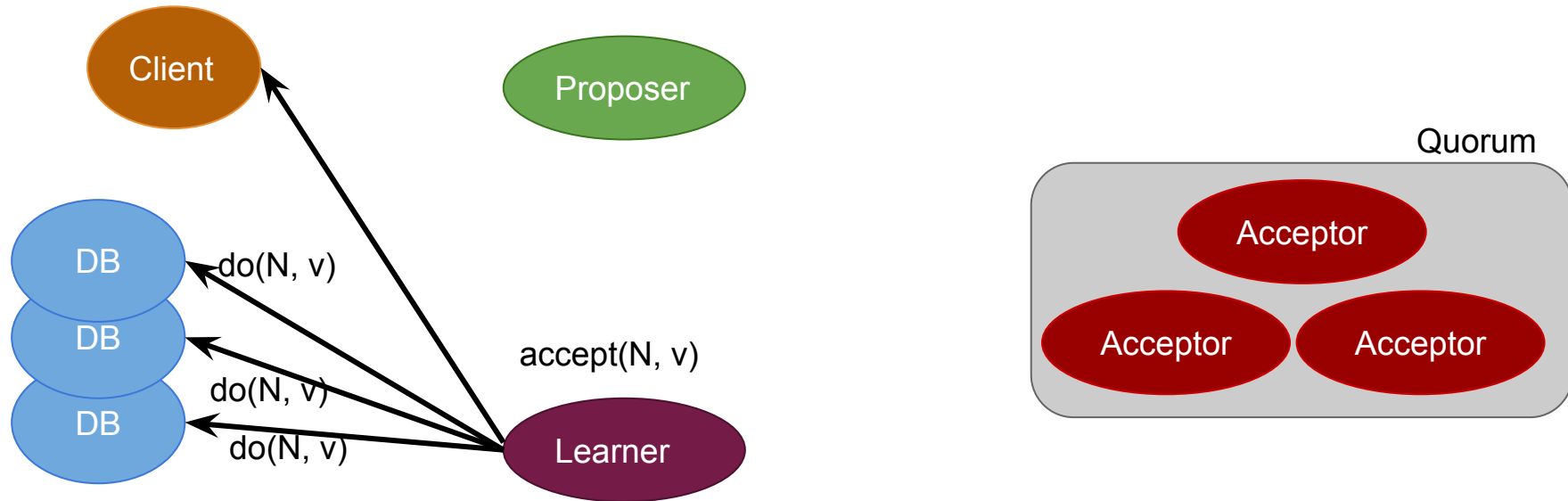
- **Acceptor:** Si la promesa aún es mantenida, anunciar el nuevo valor v
 - Enviar **accepts** a todos los Learners y al Proposer que envió el request inicial
 - No enviar **accepts** si un ID superior a N fue recibido





Paxos | Fase 2c - Accept

- **Learner:** Responde al cliente y se toman acciones respecto del valor acordado





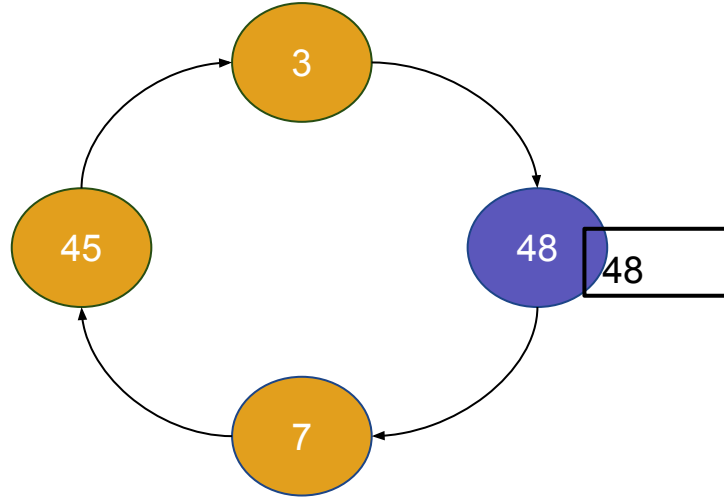
Bibliografía

- P. Verissimo, L. Rodriguez: Distributed Systems for Systems Architects, Kluwer Academic Publishers, 2001.
 - Capítulo 6: Fault-Tolerant System Foundations
 - Capítulo 7: Paradigms for Distributed Fault Tolerance
 - Capítulo 8: Models of Distributed Fault-Tolerant Computing
- G. Coulouris, J. Dollimore, t. Kindberg, G. Blair: Distributed Systems. Concepts and Design, 5th Edition, Addison Wesley, 2012.
 - Capítulo 15: Coordination And Agreement
 - Capítulo 17: Replication
- <https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>
- [The Paxos Algorithm](#)



- Raft
 - Paper
 - Diapositivas del autor del algoritmo
 - Explicación gráfica, paso a paso
 - Aplicación práctica: Etcd

Elección de Líder | Algoritmo del anillo



Ronda1: 1

Ronda2: 4

Ronda3: 4

Worst case: $N=4 \Rightarrow 3N - 1$

Best case: $2N + 1$